

Synthesis and Optimization of Digital Systems - Labs

Valentino Peluso

May 5, 2026

Contents

LAB 4 - Tutorial: Power Analysis & Optimization	1
Probabilistic Power Estimation	1
1. Extract the Switching Activity Using QuestaSIM	1
2. Switching Activity Back-Annotation into PrimeTime	2
Dynamic Power Optimization: Clock Gating	2
Introduction	2
Enabling Clock Gating	3
Leakage Power Optimization: Multi- V_{th} Synthesis	4
Post-synthesis optimization through V_{th} swapping	5
LAB 4 – Exercises	7
With this lab you learn how to:	7
Exercise 1 – Probabilistic Power Estimation with PrimeTime	7
Exercise 2 – Dynamic Power Optimization	7
Exercise 3 – Leakage-Aware Synthesis	8
Exercise 4 – Post-Synthesis Optimization	8
Description	8
Synopsis	8
Example	8

LAB 4 - Tutorial: Power Analysis & Optimization

Probabilistic Power Estimation

This section shows how to apply probabilistic power estimation to a post-synthesis netlist by extracting switching activity from simulation and back-annotating it into PrimeTime for power analysis.

1. Extract the Switching Activity Using QuestaSIM

- If not already existing, create the simulation directory:

```
mkdir ./saved/keccak_f/simulation
```

- Copy the post-synthesis Verilog netlist into the QuestaSIM working directory:

```
cp ./saved/keccak_f/synthesis/keccak_f_postsyn.v ./saved/keccak_f/simulation/.
```

- Copy the RTL testbench:

```
cp ./rtl/keccak_f/tb/keccak_tb.sv ./saved/keccak_f/simulation/.
```

- Copy the Precompiled Technology Library:

```
cp -r ./tech/STcmos65/vsimlib ./saved/keccak_f/simulation/.
```

- Change the current directory:

```
cd ./saved/keccak_f/simulation
```

- Compile the Verilog files into a new `work_swa` library:

```
vlog keccak_f_postsyn.v keccak_tb.sv -work work_swa
```

- Start the simulation of the testbench using a time resolution of 1 ns and the pre-compiled technology library `worklib`:

```
vsim -c -t ns -L vsimlib work_swa.keccak_tb -voptargs=+acc=np+keccak_f
```

- Before running the simulation, set the output destination for the VCD report and select the nets to be annotated using the following commands in the `vsim` CLI:

```
vcd files "keccak_f.vcd"
vcd add -file keccak_f.vcd -ports /keccak_tb/dut/*
vcd add -file keccak_f.vcd -internal /keccak_tb/dut/*
```

Here, the VCD commands refer explicitly to the **hierarchical structure of the testbench** used during simulation.

- **keccak_tb** is the **top-level entity (module)** of the testbench. It encapsulates the entire simulation environment, including clock and reset generation, stimulus application, and the instantiation of the design under test. In QuestaSIM, all simulated objects are referenced hierarchically starting from the top-level testbench module.
- **dut** (Design Under Test) is the **instance name of the Keccak permutation module (keccak_f) inside the testbench**. This instance represents the synthesized design whose internal activity is being observed and recorded. The instance name dut is a common convention used to clearly distinguish the design under test from the surrounding testbench logic.

As a result, the hierarchical paths: /keccak_tb/dut/ identify **all ports and internal nets of the Keccak design instance**, excluding the testbench logic. By restricting VCD dumping to this scope, the generated switching activity file contains only the signals relevant for power analysis of the synthesized design, reducing file size and avoiding unnecessary annotation of testbench-only signals.

- Run the simulation:

```
run -all
```

2. Switching Activity Back-Annotation into PrimeTime

- Go to the WORK_SYNTHESIS directory.
- Load the post-synthesis netlist in Synopsys PrimeTime:

```
pt_shell -f ./scripts/pt_analysis.tcl
```

- Annotate the switching activity using the following commands:

```
read_vcd ./saved/keccak_f/post_synthesis_sim/keccak_f.vcd \
  -strip_path /keccak_tb/dut -zero_delay

set_power_clock_scaling -period 10 [get_clock]
set power_enable_clock_scaling true
```

Dynamic Power Optimization: Clock Gating

Introduction

Clock gating is an important technique for reducing the dynamic power consumption of a design.

The figure shows a latch-based clock-gating cell and the waveforms of the signals are shown with respect to the clock signal, CLK. The clock input to the register bank, ENCLK, is gated on or off by the AND gate. ENL is the enabling signal that controls the gating. The register bank is triggered by the rising edge of the ENCLK signal.

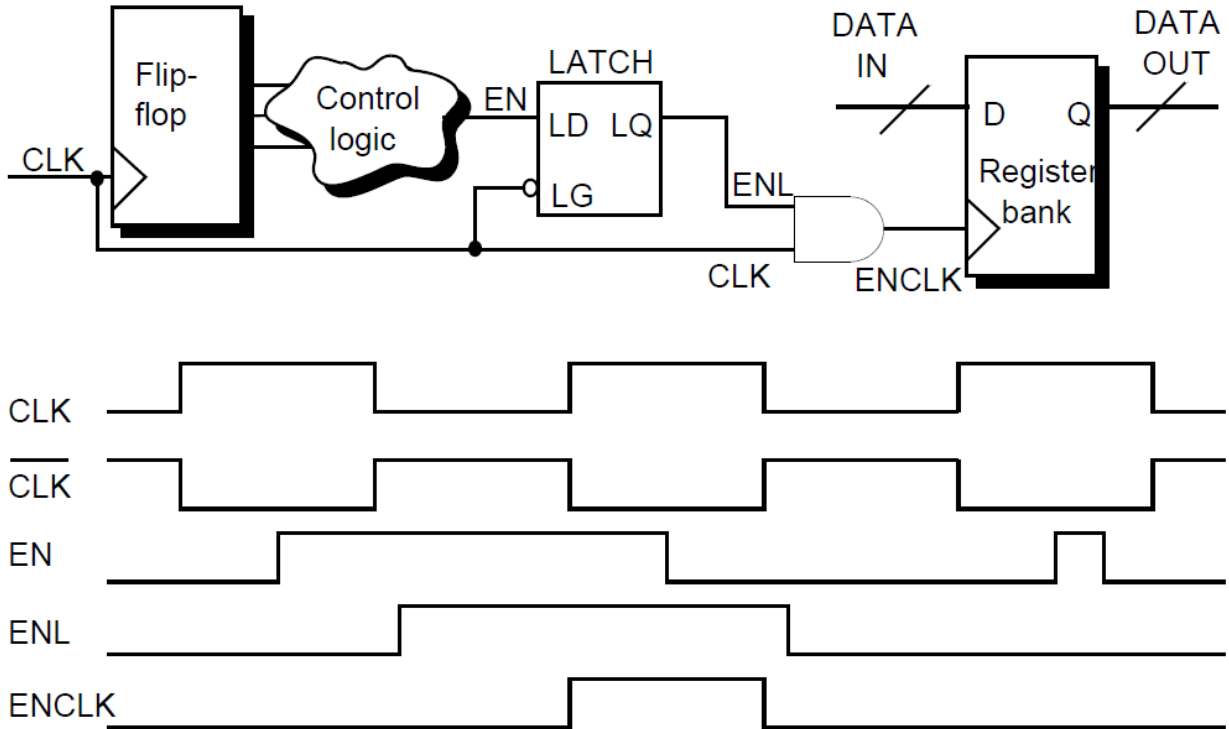


Figure 1: Clock Gating

The AND gate blocks unnecessary clock transitions by maintaining the clock signal's value after the trailing edge.

The latch prevents glitches on the EN signal from propagating to the register's clock pin. When the CLK input of the 2-input AND gate is at logic state 1, any glitching of the EN signal could, without the latch, propagate and corrupt the register clock signal. The latch eliminates this possibility because it blocks signal changes when the clock is at logic state 1.

Enabling Clock Gating

Design Compiler provides us with two commands for enabling RTL-level clock-gating: - `set_clock_gating_style`: set the clock gating style defining parameters like the bank-register width, or maximum fan-out for each clock-gating element; - `compile_ultra -gate_clock`: recognize clock-gating condition and perform the synthesis/optimization of the circuit.

In practice:

- in the scripts folder, create (if does not exist) a TCL file titled `clock_gating.tcl` with the following code:

```
set clockGateMinBitWidth 1 ;# minimum bit-width of the cg bank-register
set clockGateMaxFanout 1024 ;# maximum number of registers driven by the same cg-element

set_clock_gating_style \
  -minimum_bitwidth $clockGateMinBitWidth \
  -max_fanout $clockGateMaxFanout
```

- Copy the `synthesis.tcl` script to `synthesis_cg.tcl`.

- Source the file `clock_gating.tcl` in the `synthesis_cg.tcl` and replace the command `compile` with the following code:

```
# compile
source "../scripts/clock_gating.tcl"
compile_ultra -gate_clock
set_dont_retime [all_fanout -from [get_pins -filter is_clock_gate_output_pin] -only_cells]
```

- Comment the exit command at the end of `synthesis_cg.tcl`.
- Open the `dc_shell` and run the synthesis with clock gating:

```
cd WORK_SYNTHESIS
dc_shell-xg-t
source ../scripts/synthesis_cg.tcl
```

- Use the command `report_clock_gating` at the end of the synthesis to analyze the % of gated registers:

```
report_clock_gating
report_clock_gating -structure
report_clock_gating -enable_conditions
```

For running the STA of a design with clock-gating, we need to apply the following modifications:

- in the `synopsys_pt.setup` file replace the last line with:

```
lappend link_library [lindex $libraries 2]
```

- in the `pt_analysis.tcl` script add the `link_design` command after the `read_verilog` command:

```
read_verilog $in_verilog_filename
link_design $blockName
```

Leakage Power Optimization: Multi- V_{th} Synthesis

Multi-threshold (Multi- V_{th}) technologies provide designers with logic gates having multiple threshold voltages (V_{th}); this enables leakage power optimization. For our 65nm technology:

- `./tech/STcmos65/CORE65LPLVT_nom_1.20V_25C.db`
- `./tech/STcmos65/CORE65LPHVT_nom_1.20V_25C.db`

A common practice is to use Low- V_{th} (LVT) gates on critical paths to maintain the performance, while High- V_{th} (HVT) gates are used on non-critical paths to reduce static leakage power without incurring a delay penalty. To enable this feature into Design Compiler:

- include LVT and HVT libraries in the list of target libraries in `synopsys_dc.setup`; the resulting code should be:

```
set target_library ""
lappend target_library [lindex $link_library 3]
lappend target_library [lindex $link_library 4]
```

- include LVT and HVT libraries in the list of link libraries in `synopsys_pt.setup`: `tcl lappend link_library [lindex $libraries 3] lappend link_library [lindex $libraries 4]`
- in the file `./rtl/keccak_f/sdc/keccak_f.sdc`, replace the following line:


```
set_driving_cell -library "CORE65LPSVT_nom_1.20V_25C.db:CORE65LPSVT" -lib_cell "HS65_LS_BFX7" [all_inputs]
```

with:

```
set_driving_cell -library "CORE65LPLVT_nom_1.20V_25C.db:CORE65LPLVT" -lib_cell "HS65_LS_BFX7" [all_inputs]
```

- in the script `synthesis.tcl`, replace the following line:

```
set_operating_condition -library "${target_library}:CORE65LPSVT" nom_1.20V_25C
```

with:

```
set_operating_condition -library "[lindex $target_library 0]:CORE65LPLVT" nom_1.20V_25C
```

- replace also:

```
set_wire_load_model -library "${target_library}:CORE65LPSVT" -name area_18Kto24K [find design *]
```

with:

```
set_wire_load_model -library "[lindex $target_library 0]:CORE65LPLVT" -name area_18Kto24K [find design *]
```

- replace the command `compile` with the following lines.

```
set_attribute [find library CORE65LPLVT] default_threshold_voltage_group LVT -type string
set_attribute [find library CORE65LPHVT] default_threshold_voltage_group HVT -type string
compile_ultra
```

- In PrimeTime, use the following code for defining threshold groups:

```
set_user_attribute [find library CORE65LPLVT] default_threshold_voltage_group LVT
set_user_attribute [find library CORE65LPHVT] default_threshold_voltage_group HVT
```

- Use the command `report_threshold_voltage_group` to analyze the % of LVT/HVT cells

Post-synthesis optimization through V_{th} swapping

In Prime-Time, using the `size_cell` command we can re-size a given cell and/or map it using different V_{th} .
Example:

- the cell U101, which is a High-VT BFX2, is mapped as a Low-VT BFX2:

```
size_cell U101 CORE65LPLVT/HS65_LL_BFX2
```

- the cell U101, previously remapped with the Low-VT BFX2, is now resized as a High-VT BFX4:

```
size_cell U101 CORE65LPHVT/HS65_LH_BFX4
```

Note: Use the command `get_alternative_lib_cells` to get a collection of library cells logically equivalent to the specified cell (see man `get_alternative_lib_cells`):

```
get_alternative_lib_cells U101
```


LAB 4 – Exercises

With this lab you learn how to:

- run probabilistic power estimation
- run post-synthesis functional simulation using gate-level netlist
- extract the switching activity of internal nodes
- back-annotate switching activity into PrimeTime
- enable power features during synthesis
- clock-gating for dynamic power reduction
- multi- V_{th} for leakage power reduction
- run post-synthesis optimization through V_{th} swapping

Exercise 1 – Probabilistic Power Estimation with PrimeTime

1. Synthesize the **KECCAK-f** core with a clock period of **5 ns** using the **CORE65LPSVT** library at **nom_1.20V_25C** operating conditions.
2. Run a post-synthesis simulation and generate a **VCD** file containing the switching activity of all the input/output ports and internal nets.
3. Load the post-synthesis netlist in **PrimeTime** and report the power consumption **before** annotating the switching activity.
4. Back-annotate the switching activity into **PrimeTime**.
5. Report the power consumption **after** annotating the switching activity.
6. Compare the power consumption **before** and **after** annotating the switching activity.

Exercise 2 – Dynamic Power Optimization

Synthesize a low-power version of the **KECCAK-f** core (clock period of **5 ns** at **nom_1.20V_25C** operating conditions) enabling **dynamic power optimization only** (i.e., clock gating):

- In the `dc_shell`, analyze the percentage of gated registers.
- Open the post-synthesis Verilog netlist and check how the registers and clock-gating elements have been synthesized and connected.
- Report the power consumption using **PrimeTime**.
- Repeat the same steps with different values of:
 - `clockGateMinBitWidth = {1, 8}`
 - `clockGateMaxFanout = {8, 32, 128}`
- Report the results in: SODS 26 Group List.xlsx

Exercise 3 – Leakage-Aware Synthesis

Synthesize a low-power version of the **KECCAK-f** core (clock period **5 ns**) enabling **leakage power optimization** (i.e., Multi-Vth): - Open the post-synthesis Verilog netlist and check how many cells have been mapped to **LVT** and **HVT**. - Analyze how many cells belonging to critical paths have been mapped as **LVT** or **HVT**. - Repeat the synthesis using different clock periods: $\{1, 3\}$ ns. - Report the results in: SODS 26 Group List.xlsx

Exercise 4 – Post-Synthesis Optimization

Create and test a new command (to be run in **PrimeTime**) to swap a subset of cells to a given threshold voltage.

Description

Taking as input: 1. the list of cells to be swapped, and
2. the new threshold voltage (**LVT**, **SVT**, or **HVT**),

the command updates the netlist accordingly.

Synopsis

```
cells_swapping cellName_list Vt_type
```

Example

```
set cell_list "U300 U301 U302"  
cells_swapping $cell_list LVT
```